

PROJET DE FIN D'ÉTUDE 3A IRIS

Développement d'un système de navigation autonome pour les drones Tello

Réaliser par :

Pierre GASQUET

Côme BEAUQUIER

Dans le cadre du projet de :

DINAR FARID

Ce projet porte sur l'utilisation des drones Tello pour développer et tester un système de navigation autonome. Les drones Tello sont des drones légers et programmables, idéals pour des missions d'exploration en intérieur et en extérieur. Le projet vise à programmer des algorithmes de navigation autonome permettant aux drones d'éviter les obstacles et d'accomplir des missions comme la reconnaissance d'objets ou le suivi de trajectoires prédéfinies.

Au cours de ce projet nous avons codé plusieurs scripts nous permettant de faire des testes pour chaque parties. Tous ces codes sont dans le github du projet accessible avec le lien suivant :

Table des matières

1	Objectifs Cahier des Charges	3
2	Acquisition Prise en main du drone	3
2.1	Modèle de carénage	3
2.2	Impression 3D	4
3	Développement Implémentation d'une IHM	4
3.1	Injection de commande simple	4
3.2	IHM trajectoire	4
3.3	Problèmes rencontrés	5
4	Développement Implémentation de reconnaissance faciale	6
4.1	Modèle IA reconnaissance faciale	6
4.2	Implémentation dans le drone	6
5	Vol en essaim	7
5.1	Connexion de plusieurs drones à un même PC	8
5.2	Développement de l'application	8
6	Utilité	10
7	Conclusion	10

1 Objectifs Cahier des Charges

Nous avons défini comme objectifs les points suivants :

1. Prise en main des drones Tello et de leur SDK pour le contrôle via Python.
2. Conception et impression 3D des cadres de protection d'hélices afin de sécuriser les vols d'essai.
3. Développement et Implémentation de systèmes de navigation autonome (par exemple, pour le suivi de trajectoire ou la reconnaissance d'objets).
4. Test et validation des performances en conditions réelles.

Cahier des charges et extensions possibles :

Etude des documentations techniques

Prise en main du SDK du drone et de la communication entre le drone et l'ordinateur.

Développement d'un algorithme de navigation.

Conception et impression 3D des cadres de protection d'hélices.

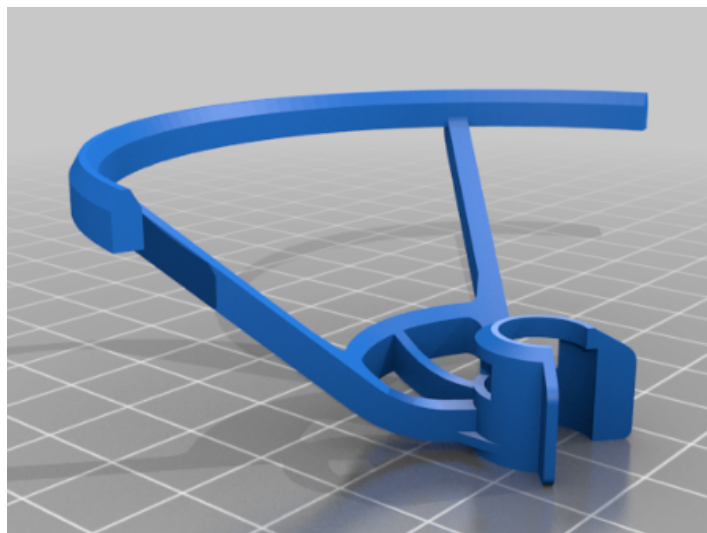
Amélioration de la détection d'obstacles avec des bibliothèques comme OpenCV.

Implémentation de systèmes avancés de reconnaissance d'objets et suivi de cibles en temps réel avec la caméra du drone.

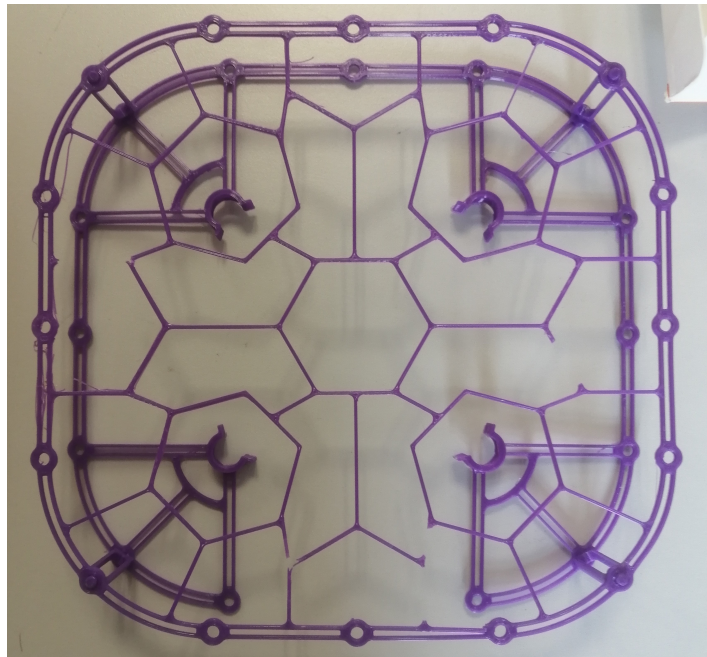
2 Acquisition Prise en main du drone

Dans un premier temps nous avons eu comme travail de trouver des modèle de protection d'hélices pour le drône que nous allions utiliser. Nous avons donc trouvé plusieurs formes pouvant correspondre à nos exigences. C'est à dire des formes simples, rapide à imprimer et légère puisque le drône a une autonomie restreinte, lui rajouter de la charge supplémentaire lui fait perdre en durée de vol. Ainsi nous avons récupéré un modèle classique de protection (fig 1) ainsi qu'un autre plus complexe mais qui a une protection quasi-intégrale mais qui reste assez fragile (fig 2).

2.1 Modèle de carénage



2.2 Impression 3D



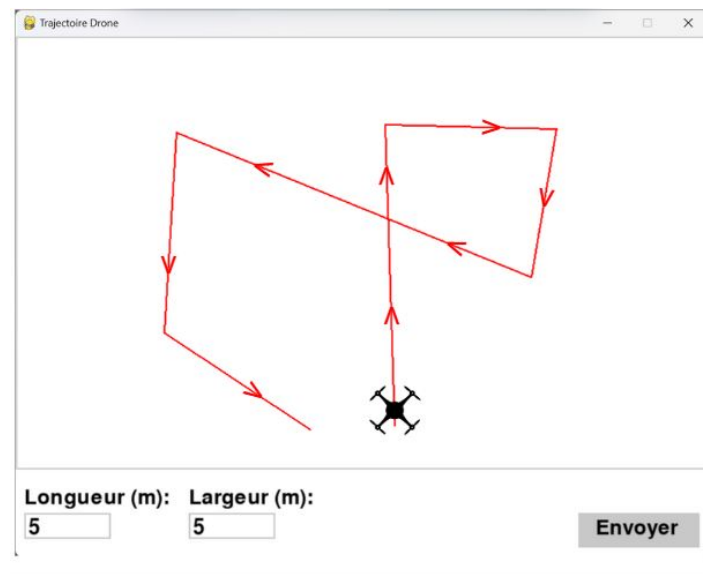
3 Développement Implémentation d'une IHM

3.1 Injection de commande simple

Dans le cadre du développement et de l'intégration des fonctionnalités du drone Tello, une première étape essentielle a consisté à tester des lignes de commande simples afin de valider la communication entre l'ordinateur et le drone. Cette démarche expérimentale permet de s'assurer que le drone répond correctement aux commandes de base, telles que le décollage (takeoff), l'atterrissage (land), le déplacement dans différentes directions (move_forward, move_back, move_left, move_right), ainsi que la gestion de la rotation (rotate_clockwise, rotate_counter_clockwise). En procédant à ces tests unitaires, il est possible d'identifier et de résoudre d'éventuels problèmes de connexion ou de latence avant d'implémenter des fonctionnalités plus complexes, comme le suivi d'objets ou la navigation autonome. Cette phase permet également de calibrer la réactivité du drone et d'ajuster les paramètres de vitesse et de déplacement en fonction de l'environnement de test.

3.2 IHM trajectoire

Dans le cadre du projet, nous avons développé une Interface Homme-Machine (IHM) permettant de définir visuellement la trajectoire du drone en traçant directement le parcours souhaité sur l'écran. Cette approche facilite la planification des déplacements du drone en remplaçant l'envoi manuel de commandes par une interaction plus intuitive. Une fois la trajectoire dessinée, les coordonnées des points de passage sont enregistrées et converties en instructions de vol que le drone exécute successivement. Cette démarche a permis de tester et d'ajuster les conversions entre les coordonnées de l'IHM et les déplacements réels du drone, en prenant en compte des facteurs tels que l'échelle, la vitesse et la précision des déplacements. L'implémentation de cette interface constitue une étape clé vers une navigation semi-autonome du drone, tout en offrant un outil de test et de validation pour l'optimisation des algorithmes de contrôle.



Lors de l'implémentation du suivi de trajectoire du drone, nous avons rencontré des difficultés liées à la gestion des changements de direction. En effet, lorsque nous traçons une trajectoire composée de deux vecteurs non colinéaires, nous avons observé que le drone ne pivotait pas sur lui-même avec la commande `rotate_clockwise` avant de poursuivre son déplacement. Au lieu de cela, il évoluait en effectuant des mouvements segmentés, se déplaçant d'abord en ligne droite sur un axe, puis sur un autre, formant ainsi une trajectoire en marche par rectangle au lieu d'un parcours fluide. Cette anomalie était due à une mauvaise gestion de l'orientation du drone entre deux segments, qui ne prenait pas en compte l'angle nécessaire pour aligner la direction du drone avec la nouvelle trajectoire. Pour corriger cela, nous avons dû intégrer un calcul d'angle dynamique permettant au drone d'effectuer une rotation appropriée avant de poursuivre son déplacement, assurant ainsi une trajectoire plus naturelle et conforme au tracé initial.

3.3 Problèmes rencontrés

Lors de l'implémentation du suivi de trajectoire du drone, nous avons rencontré des difficultés liées à la gestion des changements de direction. En effet, lorsque nous traçons une trajectoire composée de deux vecteurs non colinéaires, nous avons observé que le drone ne pivotait pas sur lui-même avec la commande `rotate_clockwise` avant de poursuivre son déplacement. Au lieu de cela, il évoluait en effectuant des mouvements segmentés, se déplaçant d'abord en ligne droite sur un axe, puis sur un autre, formant ainsi une trajectoire en marche par rectangle au lieu d'un parcours fluide. Cette anomalie était due à une mauvaise gestion de l'orientation du drone entre deux segments, qui ne prenait pas en compte l'angle nécessaire pour aligner la direction du drone avec la nouvelle trajectoire. Pour corriger cela, nous avons dû intégrer un calcul d'angle dynamique permettant au drone d'effectuer une rotation appropriée avant de poursuivre son déplacement, assurant ainsi une trajectoire plus naturelle et conforme au tracé initial.

```
# Calcul de la distance et de l'angle
distance = math.sqrt(dx**2 + dy**2) # Distance entre les points
alpha = math.degrees(math.atan2(dx, dy)) # Angle cible en degrés
print(f"distance = {distance}, alpha (angle cible) = {alpha}")

# Calcul de l'angle de rotation nécessaire
rotation_angle = angle - alpha
if rotation_angle > 180:
    rotation_angle -= 360
elif rotation_angle < -180:
    rotation_angle += 360
```

4 Développement Implémentation de reconnaissance faciale

4.1 Modèle IA reconnaissance faciale

Dans la continuité du projet, nous avons intégré un modèle d'intelligence artificielle de reconnaissance faciale, ResNet-10 SSD, afin de permettre au drone de suivre un visage en temps réel. Avant son implémentation sur le drone, nous avons d'abord testé ce modèle sur un ordinateur pour valider son bon fonctionnement et évaluer ses performances en termes de précision et de rapidité de détection. Cette phase de test nous a permis d'optimiser les paramètres du modèle, tels que le seuil de confiance pour la détection des visages, ainsi que d'assurer la compatibilité avec OpenCV et les ressources matérielles disponibles. Une fois ces validations effectuées, nous avons intégré le modèle au système du drone et adapté le traitement des images en fonction du flux vidéo fourni par sa caméra embarquée. Cette approche progressive a garanti une transition fluide entre les tests sur PC et l'implémentation en conditions réelles, permettant ainsi un suivi dynamique et efficace du visage détecté par le drone. Lorsque le modèle d'intelligence artificielle reconnaît un visage, il crée un rectangle autour de celui-ci.

4.2 Implémentation dans le drone

L'objectif est maintenant de faire en sorte que le drone suive le visage reconnu grâce à la reconnaissance faciale décrite ci-dessus. Pour cela, nous devons nous assurer que le rectangle de reconnaissance faciale reste toujours au centre de l'image capturée par le drone.

Tout d'abord, nous récupérons les images prises par la caméra du drone et détectons les visages présents. Nous sélectionnons ensuite le visage le plus proche du drone (c'est-à-dire celui dont le rectangle de détection a la plus grande surface). Une fois le centre de ce rectangle déterminé, nous ajustons la vitesse du drone à l'aide de la fonction `send_rc_control()`. Cette fonction permet de modifier les différentes vitesses du drone :

- La vitesse de déplacement « Gauche - Droite ».
- La vitesse de déplacement « Avant - Arrière ».
- La vitesse d'ascension.
- La vitesse de rotation.

Dans notre cas, nous agissons sur la vitesse de déplacement « Avant - Arrière », la vitesse d'ascension et la vitesse de rotation.

Pour calculer ces vitesses, nous mesurons la différence entre le centre de l'image et le centre du rectangle détecté selon les axes x et y :

- La vitesse d'ascension est proportionnelle à la différence selon y .
- La vitesse de rotation est proportionnelle à la différence selon x .

Concernant la vitesse de déplacement « Avant - Arrière », nous calculons la différence entre la largeur du rectangle détecté et une largeur de référence (fixée à 100 pixels dans notre cas). La vitesse est alors proportionnelle à cette différence.

Avec cet algorithme, le drone suit correctement le visage, mais un problème subsiste : les images reçues sont saccadées, car le temps de calcul et d'envoi des commandes est trop long. En effet, le temps de calcul et d'envoi des commandes au drone est suffisamment long pour provoquer ces saccades. Afin d'obtenir un résultat plus fluide, nous allons exécuter en multithreading la fonction qui calcule et envoie les commandes au drone. Cela permettra de paralléliser l'envoi des commandes et l'exécution du reste du programme.



FIGURE 1 – Drone en vol avec reconnaissance faciale

Applications

Nous pouvons maintenant envisager différentes applications possibles pour ce programme :

- **Drone de loisir** : Une première application consiste simplement à suivre une personne souhaitant se filmer avec un drone, sans avoir besoin de le piloter manuellement. Cela peut être particulièrement utile pour des activités sportives.
- **Drone de surveillance** : En cas d'intrusion dans une zone interdite, un drone pourrait être déployé pour surveiller et suivre l'intrus en temps réel.
- **Drone de sauvetage** : Le drone pourrait se rendre auprès d'une personne en détresse et la suivre automatiquement, fournissant des informations cruciales sur l'évolution de la situation avant l'arrivée des secours.

5 Vol en essaim

Nous allons maintenant développer une application permettant de contrôler un essaim de drones. Le but de cette application est d'envoyer plusieurs drones effectuer une action à des coordonnées définies par l'utilisateur. Lorsque plusieurs drones sont en vol, chacun pourra traiter des points différents de manière autonome.

5.1 Connexion de plusieurs drones à un même PC

Afin de connecter plusieurs drones à un même ordinateur, nous devons contourner une limitation technique. En effet, jusqu'à présent, nous avons connecté un drone en utilisant le signal Wi-Fi qu'il émet. Cependant, cette méthode ne fonctionne pas lorsque plusieurs drones doivent être connectés simultanément.

Pour résoudre ce problème, nous allons configurer chaque drone pour qu'il se connecte à un partage de connexion créé sur notre ordinateur.

La procédure est la suivante : nous nous connectons normalement à chaque drone, puis nous lui envoyons les instructions nécessaires pour qu'il rejoigne le réseau Wi-Fi du partage de connexion à l'aide de l'application PacketSender.

PacketSender est un logiciel permettant d'envoyer des paquets de données sur un réseau. Nous utilisons cette application pour transmettre les instructions suivantes aux drones :

- `command` : Met le drone en mode réception, lui permettant d'accepter les commandes suivantes. Cette étape revient à "déverrouiller" le drone.
- `battery` : Permet de récupérer le niveau de batterie du drone.
- `ap NomDuReseau MotDePasseDuReseau` : Indique au drone qu'il doit désormais tenter de se connecter au réseau `NomDuReseau` en utilisant le mot de passe `MotDePasseDuReseau`.

Une fois ces instructions envoyées, le drone redémarre et se connecte automatiquement au partage de connexion. Cette manipulation doit être effectuée pour chaque drone afin de pouvoir contrôler l'ensemble de l'essaim depuis le même ordinateur.

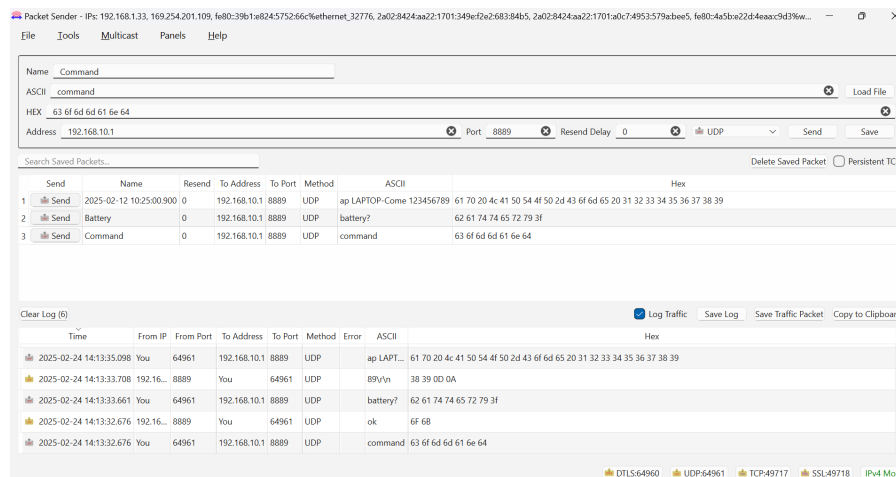


FIGURE 2 – Application PacketSender

5.2 Développement de l'application

Pour cette application, nous allons tout d'abord créer une interface Homme-Machine permettant à l'utilisateur de sélectionner les points où les drones devront se rendre. Cette interface sera développée à l'aide de la bibliothèque Pygame.

L'utilisateur pourra lancer le programme et indiquer les différents points en cliquant sur l'interface. Les points et leurs coordonnées seront alors stockés dans une liste.

Notre algorithme fera ensuite appel à la fonction `send_drone_to_points()`. Cette fonction, pour chaque drone, appellera la fonction `move_to_point()`, qui sélectionnera le premier point de la liste, le supprimera, orientera le drone vers la cible, puis le fera avancer vers ce point. Dès

qu'un drone atteint sa destination, sa position, son angle et son état de vol seront enregistrés dans la liste d'objets `drones_states`. Ces informations permettront de calculer l'angle de rotation et la distance à parcourir pour atteindre le point suivant. Tant que la liste des points n'est pas vide, la fonction est rappelée.

Enfin, une fois que tous les points ont été atteints par les drones, nous calculons l'angle et la distance nécessaires pour qu'ils retournent à leur zone de décollage.

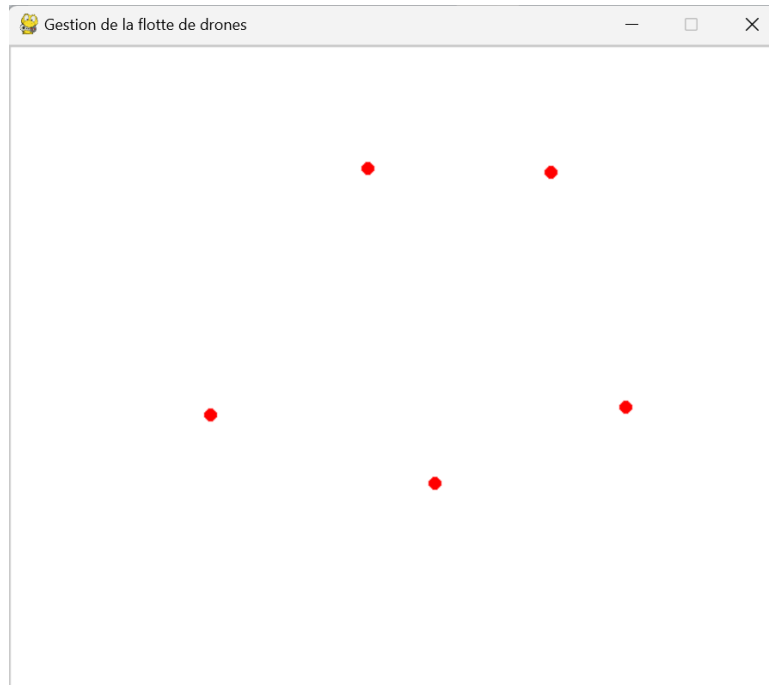


FIGURE 3 – Interface homme machine pour l'essaim de drone

Applications

Ce programme peut être soumis à différents types d'applications.

- **Drones pour la surveillance** : Notre essaim de drones peut rapidement se rendre à des points spécifiques d'une zone pour prendre des photos et surveiller les environs.
- **Drones pour la dépollution des océans** : L'essaim de drones peut être utilisé pour ramasser les plastiques à la surface de l'océan. En fournissant la position de chaque déchet à ramasser, les drones peuvent se rendre au-dessus et procéder à leur collecte.
- **Drones pour la cartographie** : Notre essaim de drones pourrait collecter des données à différents points d'une zone afin de créer des cartes ou d'utiliser les informations pour des applications scientifiques définies.
- **Drones pour l'agriculture** : L'essaim de drones peut également être utilisé, par exemple, pour planter des graines à des intervalles précis dans le cadre d'une exploitation agricole.

6 Utilité

L'utilisation d'un essaim de drones autonomes dans une zone définie par une Interface Homme-Machine (IHM) ouvre de nombreuses perspectives dans des domaines variés tels que la sécurité, la surveillance, la logistique et la recherche scientifique. En permettant aux drones de se déplacer vers des points spécifiques commandés via l'IHM, il devient possible d'optimiser des tâches telles que la détection de visages et le suivi d'objets. Cette approche offre un gain de rapidité et d'efficacité par rapport à un drone unique, notamment pour la surveillance de vastes zones en temps réel ou la coordination d'actions simultanées. Dans le cadre de la reconnaissance faciale, un essaim de drones peut être utilisé pour identifier et suivre des individus dans des environnements complexes, garantissant une couverture plus large et réduisant les angles morts. De plus, dans des applications telles que la gestion de foules, la sécurité d'événements, ou les missions de sauvetage, ces drones autonomes peuvent se repositionner dynamiquement en fonction des données transmises, assurant ainsi une réactivité accrue face aux situations évolutives. L'intégration d'une telle technologie représente une avancée majeure vers des systèmes de surveillance intelligents, flexibles et autonomes, capables d'interagir en temps réel avec leur environnement et les opérateurs humains.

7 Conclusion

Ce projet a permis de prendre en main les drones Tello et leur SDK afin de les contrôler via Python, en assurant une communication fluide entre le drone et l'ordinateur. Nous avons d'abord testé des commandes simples pour valider le bon fonctionnement des déplacements, puis nous avons conçu et développé une Interface Homme-Machine (IHM) permettant de dessiner une trajectoire que le drone pouvait suivre de manière autonome. Cette IHM a nécessité l'implémentation d'un algorithme de navigation prenant en compte les variations d'orientation du drone pour éviter des trajectoires non optimales, comme la marche en rectangle.

En parallèle, nous avons intégré des cadres de protection d'hélices imprimés en 3D, afin de sécuriser les vols d'essai et réduire les risques d'endommagement des drones. Ces tests ont permis de valider les performances en conditions réelles, d'ajuster les paramètres de vitesse et de stabilisation du drone, et d'optimiser son comportement face aux imprécisions de navigation.

Nous avons ensuite développé un système de reconnaissance faciale basé sur ResNet-10 SSD puis MobileNet SSD, en testant d'abord ces modèles sur PC avant de les implémenter sur le drone. Ce système a été combiné avec un algorithme de suivi d'objets en temps réel, permettant au drone d'ajuster sa position en fonction de la cible détectée. Une problématique clé a été la gestion des pertes de suivi, que nous avons améliorée en intégrant une logique de recherche automatique permettant au drone de retrouver la cible lorsqu'elle sort temporairement de son champ de vision.

Enfin, nous avons exploré la possibilité d'utiliser un essaim de drones autonomes, capables de se déplacer vers des points commandés via l'IHM pour exécuter des missions telles que la surveillance ou la reconnaissance faciale en temps réel. Cette approche ouvre des perspectives d'optimisation dans les domaines de la sécurité, la logistique et l'exploration automatisée.

En respectant le cahier des charges, nous avons ainsi réussi à développer un algorithme de navigation, implémenter un système avancé de suivi de cibles, et améliorer la sécurité des vols tout en explorant des extensions potentielles telles que la reconnaissance d'objets en temps réel. Ce projet constitue une base solide pour des développements futurs, notamment en matière de détection d'obstacles et d'intelligence collective entre drones.